Laboratório Funcional · Aula 12

Matheus Goulart de Alencar · RA 24131995-2 · Turma ESOFT 6SB
Gerado em 16/10/2026, 20:02:01 · tempo registrado no cronômetro: 49:24

Preparação: Programar com funções matemáticas

def area(r): return 3.14159 * r * r

Pura

def agora(): return datetime.now()

Impura

def total(lista): return sum(lista)

Pura

def registra(msg): print(msg)

Impura

def sorteia(): return random.randint(1, 6)

Impura

def poe(lista, x): lista.append(x); return lista

Impura

0.1 Escolha duas funções que você marcou como impuras e explique, para cada uma, o que a impede de ser pura.

A segunda é impura pois não retorna sempre o mesmo resultado e acaba dependendo de um fator externo

A quarta é impura pois produz efeito colateral

A quinta não retorna sempre o mesmo resultado e depende de um fator aleatório

A sexta também produz um efeito colateral e não preserva os dados de entrada

Prática 1: Funções puras e formas funcionais

Previsão: O que as linhas 1, 4 e 7 vão imprimir?

1: A-1 / A-2

4: 14

7: [7, 3, 9] [3, 7, 9]

Saídas (original e com compor(g, f))

```
1: A-1 A-2
2: False
3: 8
4: 18
5: [4, 9, 16]
6: 29
```

```
7: [7, 3, 9] [3, 7, 9]
8: [3, 7, 9]
```

1.1 Por que proximo_id("A") devolve valores diferentes para a mesma entrada? Por que isso dificulta os testes (p. 637)?

A função proximo_id("A") devolve valores diferentes pois depende da variável global contador, que é incrementada a cada chamada. Com isso, a primeira chamada retorna "A-1" e a segunda retorna "A-2", mesmo recebendo a mesma entrada.

Isso dificulta os testes pois o resultado depende do estado anterior do programa. Para testar a função, é necessário considerar o valor atual de contador e a ordem das chamadas, isso pode tornar os testes menos previsíveis e reproduzíveis.

1.2 compor(f, g)(4) e compor(g, f)(4) deram valores diferentes. Calcule os dois à mão e explique a diferença.

A explicação é pois a ordem das operações altera o resultado, na primeira composição multiplicamos por 3 e depois somamos 2, já na segunda somamos 2 e depois multiplicamos por 3.

1.3 Qual é a diferença entre sorted(notas) e notas.sort()? Qual dos dois segue o estilo funcional, e por quê?

A diferença é que sorted(notas) cria uma nova lista ordenada sem modificar a lista original, enquanto notas.sort() ordena a própria lista alterando elas.

Prática 2: Scheme: tudo é lista

Previsão: (car '(A B C))

(car '(A B C)) -> A

Previsão: (cdr '(A B C))

(cdr '(A B C)) -> (B C)

Previsão: (cons '(A B) '(C D))

(cons '(A B) '(C D)) -> ((A B) C D)

Saídas (original e sem o apóstrofo)

```
Com apóstrofo: A

Sem apóstrofo: Error: A is not defined
```

2.1 Para que serve o apóstrofo (')? O que o Scheme tentou fazer com (A B C) quando ele não estava lá?

O apóstrofo ' serve para indicar que a expressão deve ser tratada como dado, sem ser executada como código.

Sem o apóstrofo, o Scheme tentou executar (A B C) como uma chamada de função, considerando A como o nome da função, como A não está definida ocorreu um erro

2.2 Escreva uma expressão que devolva o terceiro elemento de '(A B C) usando só car e cdr. Teste no OneCompiler e cole aqui a expressão e o resultado.

```
(car (cdr (cdr '(A B C)))) → C
```

Prática 3: Recursão e recursão em cauda

O que vi no passo a passo

n = 4: comum em 8 passos, até 3 multiplicações esperando (expressão de até 30 caracteres); em cauda em 6 passos, nada esperando (até 17 caracteres).

Previsão: O programa em Scheme chama conta-ate recursivamente um milhão de vezes. Vai funcionar? E a mesma função em Python?

Não vai funcionar em Scheme se houver limite de profundidade da pilha, pois um milhão de chamadas recursivas pode exceder esse limite, só funcionará se a implementação suportar essa recursão, por exemplo, com otimização de chamadas em cauda.

Em Python, a função também não funcionará normalmente: o limite padrão de recursão é muito menor que um milhão, então ocorrerá um RecursionError.

Saída do Scheme

```
#t
#f
2432902008176640000
2432902008176640000
1000000
```

Saída do Python (as últimas linhas bastam)

```
500
RecursionError: maximum recursion depth exceeded
```

3.1 No passo a passo, o que cresce na recursão comum e não cresce na recursão em cauda? Por quê?

Na recursão comum, crescem os registros de ativação na pilha de execução, pois cada chamada precisa esperar a chamada seguinte terminar para realizar a multiplicação.

Na recursão em cauda, não fica nenhuma operação pendente após a chamada recursiva. Por isso, linguagens que otimizam chamadas em cauda podem reutilizar o mesmo registro de ativação, mantendo o uso de memória constante.

3.2 Por que o Scheme fez um milhão de chamadas e o Python parou com RecursionError? (Dica: releia o quadro sobre a otimização da chamada em cauda e a mensagem de erro do Python. O livro fala do Scheme na p. 656.)

O Scheme conseguiu fazer um milhão de chamadas porque otimiza a recursão em cauda, reutilizando o registro de ativação e evitando o crescimento da pilha de execução.

Já o Python não faz essa otimização automaticamente e possui um limite de profundidade de recursão. Por isso, ao ultrapassar esse limite, ocorre o erro RecursionError.

Prática 4: Mão na massa: três funções sobre listas

Saída final (com os três ok)

```
tamanho: 3  ok
soma-quadrados: 14  ok
```

```
pares: (2 4 6)  ok
```

4.1 Cole aqui as suas três funções (tamanho, soma-quadrados e pares).

```
(define (tamanho lis)
  (if (null? lis)
      0
      (+ 1 (tamanho (cdr lis)))))

(define (soma-quadrados lis)
  (if (null? lis)
      0
      (+ (* (car lis) (car lis))
         (soma-quadrados (cdr lis)))))

(define (pares lis)
  (cond ((null? lis) '())
        ((even? (car lis))
         (cons (car lis) (pares (cdr lis))))
        (else (pares (cdr lis)))))
```

4.2 Qual foi a parte mais difícil? Onde você usou null?, car e cdr em cada função?

A parte mais difícil foi entender como a recursão percorre a lista até chegar ao caso vazio.

- tamanho: usei null? para verificar se a lista está vazia, car não foi necessário e cdr para continuar contando os elementos restantes.

- soma-quadrados: usei null? para identificar o fim da lista, car para obter o primeiro número e calcular seu quadrado, e cdr para somar os quadrados dos elementos restantes.

- pares: usei null? para verificar o fim da lista, car para verificar se o primeiro número é par com even?, e cdr para continuar percorrendo a lista. Usei cons para construir a lista com os números pares encontrados.

Prática 5: Haskell: padrões, compreensões e preguiça

Previsão: squares é uma lista infinita. O que você acha que take 5 squares imprime? E primeiro 42 (div 1 0)?

take 5 squares: imprime os cinco primeiros quadrados da lista infinita: [0,1,4,9,16].

primeiro 42 (div 1 0): devolve 42, porque a função primeiro ignora o segundo parâmetro (_). Pela avaliação preguiçosa do Haskell, a divisão por zero não é calculada.

Saídas (original e com member 17)

```
Original:

2432902008176640000
[1,2,4,7,14]
[2,2,4,7,8,9]
[0,1,4,9,16]
[0,1,4,9,16,25,36,49]
True
42
```

```
Com member 17:

2432902008176640000
[1,2,4,7,14]
[2,2,4,7,8,9]
[0,1,4,9,16]
[0,1,4,9,16,25,36,49]
True
42
```

5.1 Como o Haskell consegue imprimir parte de uma lista infinita? E por que primeiro 42 (div 1 0) não deu erro de divisão por zero?

O Haskell consegue imprimir parte de uma lista infinita por causa da avaliação preguiçosa: ele calcula apenas os elementos necessários, por exemplo, take 5 squares calcula somente os cinco primeiros quadrados.

Já primeiro 42 (div 1 0) não dá erro porque a função primeiro ignora o segundo parâmetro (_), como o Haskell só calcula uma expressão quando seu valor é necessário, a divisão por zero nunca é executada e o resultado é 42.

5.2 O que aconteceu com member 17 squares? Por que ele nunca termina, enquanto member 16 squares terminou? Como você corrigiria? (Dica: squares está em ordem crescente. Quando já dá para desistir da busca? O livro mostra uma solução na p. 670.)

member 17 squares nunca termina porque 17 não é um quadrado perfeito, como squares é uma lista infinita, a função continua procurando indefinidamente.

Já member 16 squares termina porque encontra o número 16 na lista e retorna True.

Como corrigir: como a lista está em ordem crescente, podemos parar a busca quando o elemento atual for maior que o número procurado.

```
member b [] = False
member b (a:x)
 | a > b    = False
 | a == b   = True
 | otherwise = member b x
```

Assim, member 17 squares termina quando encontra 25, pois 25 > 17, e retorna False.

Prática 6: Você já usa funcional

Previsão: O que mais10(5) e [1, 2, 3].map(vezes(5)) vão imprimir?

mais10(5) imprime 15, pois a função soma 10 ao valor recebido.

[1, 2, 3].map(vezes(5)) imprime [5, 10, 15], pois vezes(5) devolve uma função que multiplica cada elemento por 5.

Saída do JavaScript

```
15
[ 5, 10, 15 ]
56
{ nome: 'Ana', nota: 7 } { nome: 'Ana', nota: 8 }
```

Saída do Java

```
Segue o print
```




```java
import java.util.*;
import java.util.function.*;

public class Main {
    record Produto(String nome, double preco) {}

    public static void main(String[] args) {
        Function<Integer, Integer> f = x -> x + 2;
        Function<Integer, Integer> g = x -> 3 * x;
        System.out.println(f.compose(g).apply(4));     // f(g(4))
        System.out.println(f.andThen(g).apply(4));     // g(f(4))

        List<Produto> produtos = List.of(new Produto("Teclado", 150), new Produto("Mouse", 80),
                                         new Produto("Monitor", 900), new Produto("Cabo", 35));
        double caros = produtos.stream()
            .filter(p -> p.preco() > 100)
            .mapToDouble(Produto::preco)
            .sum();
        System.out.println(caros);
        System.out.println(produtos.stream().map(p -> p.nome().toUpperCase()).sorted().toList())
    }
}
```

6.1 somador(10) já terminou de executar quando mais10(5) é chamado. Como a função devolvida ainda "lembra" o 10?

A função devolvida ainda “lembra” o valor 10 porque é um fechamento (closure), ela mantém uma referência ao ambiente em que foi criada, incluindo a variável n, cujo valor é 10, assim, mesmo depois que somador(10) termina, mais10(5) consegue usar esse valor e retornar 15.

6.2 Por que f.compose(g).apply(4) e f.andThen(g).apply(4) deram resultados diferentes? Qual deles corresponde ao compor(f, g) da Prática 1?

Os resultados foram diferentes porque a ordem de execução das funções muda.

f.compose(g).apply(4) executa primeiro g(4) = 12 e depois f(12) = 14.

f.andThen(g).apply(4) executa primeiro f(4) = 6 e depois g(6) = 18.

Prática 7: O laço vira pipeline

7.1 Primeiro exercício escolhido (1, 2 ou 3): cole o seu código reescrito.

```
1° Ex escolhido:

import java.util.*;

public class Main {
    public static void main(String[] args) {
        List<Integer> nums = List.of(3, 8, 5, 12, 7, 6);

        int soma = nums.stream()
            .filter(n → n % 2 == 0)
            .mapToInt(n → n * n)
            .sum();
```

```
        System.out.println(soma);
    }
}
```

7.2 Segundo exercício escolhido: cole o seu código reescrito.

```
2° Ex escolhido:

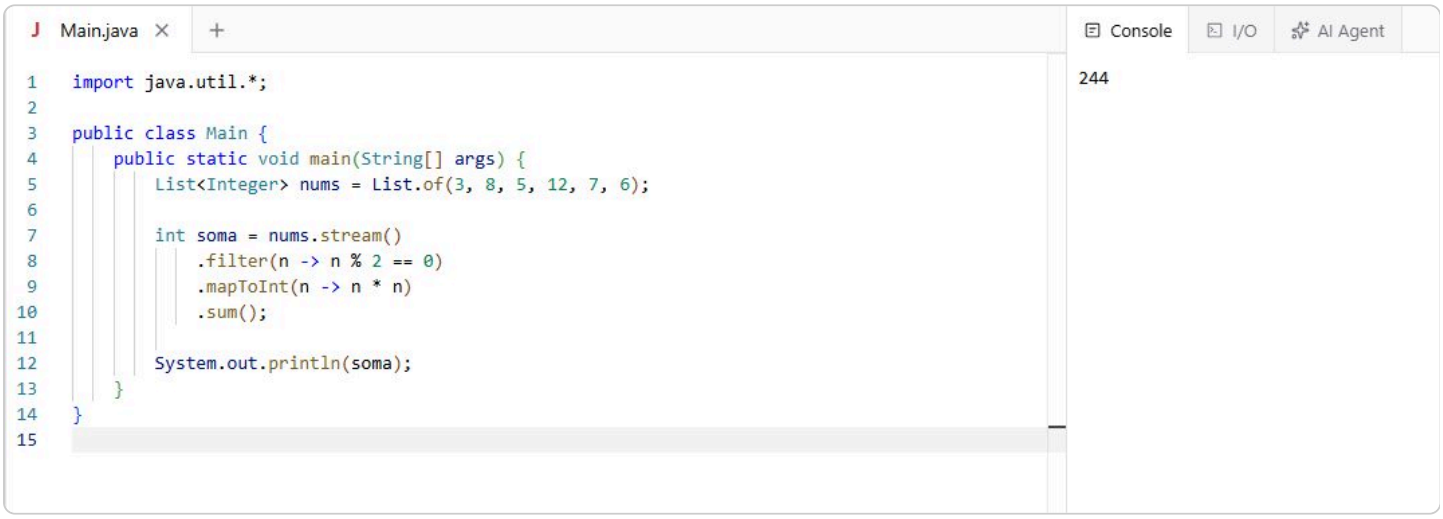
const alunos = [
  { nome: "Ana", nota: 8.5 },
  { nome: "Bruno", nota: 5.0 },
  { nome: "Carla", nota: 7.0 },
  { nome: "Davi", nota: 9.5 },
];

const aprovados = alunos
  .filter((aluno) ⇒ aluno.nota ⩾ 6)
  .map((aluno) ⇒ aluno.nome.toUpperCase());

console.log(aprovados.join(", "));
```

Saídas das suas versões

```
Segue o print
```





7.3 Qual versão é mais fácil de ler: a com laço ou a funcional? E qual é mais fácil de depurar (por exemplo, para ver um valor intermediário)?

A versão funcional é mais fácil de ler, pois o código fica mais curto e organizado, mostrando diretamente o que queremos fazer com os dados.

Já a versão com laço é mais fácil de depurar, pois permite verificar os valores a cada repetição e identificar mais facilmente onde ocorre um erro.

Fechamento: O que fica do paradigma funcional

Uma função pura pode ler uma variável global que muda.

Falso

Em Scheme, a recursão em cauda não faz a pilha crescer.

Verdadeiro

O Python otimiza a recursão em cauda.

Falso

Em Haskell, [1 ..] só é calculada à medida que é usada.

Verdadeiro

filter, map e reduce dispensam laço e variável acumuladora.

Verdadeiro

Currying transforma uma função de vários parâmetros numa cadeia de funções de um parâmetro.

Verdadeiro

8.1 Qual versão é mais fácil de entender, e por quê? O livro estima que um programa funcional costuma levar cerca de 2 vezes mais tempo para executar (p. 678). Em que tipo de sistema você aceitaria esse custo, e em qual não?

A versão em Haskell é mais fácil de entender, pois é mais curta e expressa diretamente as operações realizadas, sem precisar de uma variável acumuladora e de um laço explícito.

Eu aceitaria o custo de execução maior em sistemas nos quais a clareza e a facilidade de manutenção são mais importantes, como ferramentas de análise de dados, já em sistemas que exigem alto desempenho e respostas rápidas, como jogos e sistemas de tempo real, preferiria uma versão mais eficiente em C.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

O que mais me surpreendeu nesta atividade foi o entendimento que obtive sobre as funções de diferentes linguagens que foram passadas, tornando-as mais fáceis.

8.3 Qual é o erro de impressão, e como corrigir? O que a mensagem do compilador diz sobre o tipo que ele inferiu?

O erro de impressão está em b <- h, pois h é um único elemento, e não uma lista, o correto é b <- h, para filtrar os elementos menores ou iguais ao pivô. Assim, a linha fica:

```
sort (h:t) = sort [b | b <- t, b <= h]
        ++ [h] ++
        sort [b | b <- t, b > h]
```

A mensagem do compilador indica que ele esperava uma lista, mas encontrou um valor individual, evidenciando um conflito de tipos.